

4. Architecture Note

ParcelPilot AI Agent – Architecture

ParcelPilot is a tool-oriented support agent that resolves customer and internal support requests by combining structured business data with policy documentation, rather than relying solely on document retrieval or a raw LLM response.

1. Agent Design

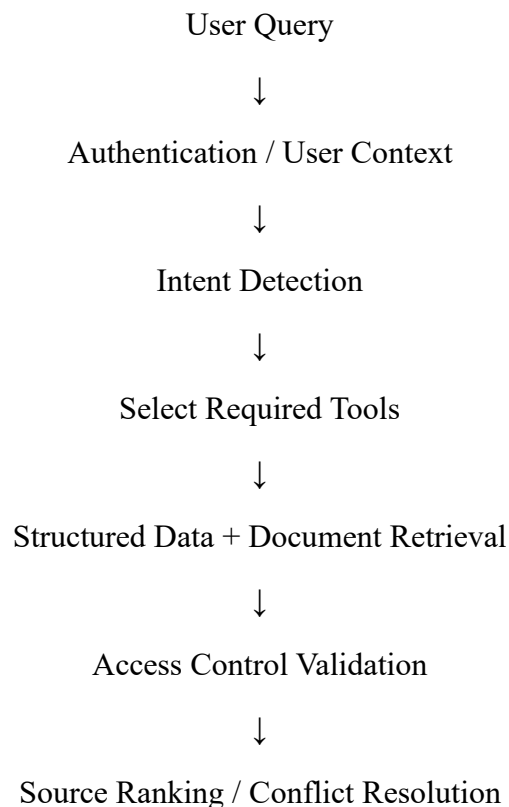
The main entry point is:

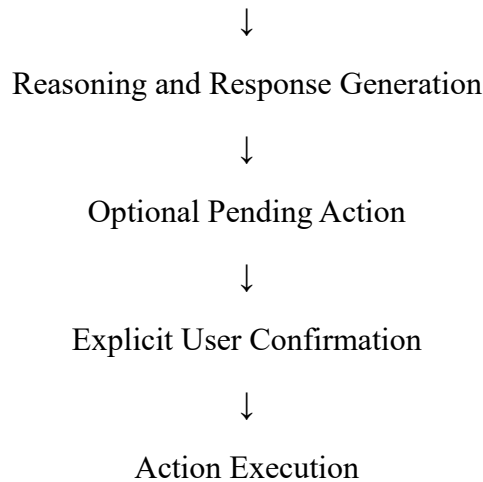
`AgentService.handle_query(query, user_context)`

The agent handles several categories of requests, including:

- Order status and cancellation queries
- Service credit eligibility
- Ticket investigation
- Product known issues
- Account-specific policy questions
- Escalation requests
- Explicit confirmation of pending actions

The agent follows a controlled workflow:





A key design decision was to separate reading information from performing state-changing actions. For example, when a user asks:

"Please escalate order ORD-1001 to the support team."

the system does not immediately execute the escalation. Instead, it creates a pending action and returns an action ID:

ACT-XXXX

Status: PENDING_CONFIRMATION

The user must explicitly confirm the action:

Confirm ACT-XXXX

Only then is the escalation executed. This prevents accidental state-changing operations and provides a safer workflow.

2. Tool Design

The system is divided into separate services, each responsible for a specific capability.

AgentService is the orchestration layer. Its responsibilities include processing natural-language queries, detecting the relevant intent, calling structured-data and document retrieval services, applying account access rules, resolving policy conflicts, and preparing state-changing actions that require explicit confirmation before execution. The agent coordinates the other services rather than putting all logic into a single component.

DataService handles structured business data — accounts, orders, tickets, pickup timing, cancellation timing, and account-specific orders and tickets. Supported operations include `get_order()`, `get_order_for_user()`, `get_account()`, `get_account_for_user()`, `get_ticket()`, `get_ticket_for_user()`, `get_pickup_delay_minutes()`, and `get_cancellation_minutes()`. This lets the agent calculate from actual structured data rather than inferring values from documents — for example, the service credit workflow uses scheduled pickup window, actual pickup time, carrier fault, customer fault, shipment fee, and account-specific contract terms.

DocumentService retrieves information from ParcelPilot documentation — current support policies, cancellation and service credit SOPs, product operations documentation, known issues, customer-specific agreements, and deprecated policies. For a query like *"Is there a known issue with bulk CSV upload failing for large shipment files?"*, the agent searches the knowledge base and retrieves the relevant documentation — in this dataset, **KI-208 (Bulk Upload failures on large CSVs)**, which explains that some Growth and Enterprise customers see intermittent failures above ~3,000 rows despite a supported limit of 5,000, with a documented workaround of splitting uploads below ~3,000 rows.

ActionService handles state-changing operations around a confirmation workflow:

Create Action → PENDING_CONFIRMATION → Return Action ID →

User Explicitly Confirms → Validate User Ownership →

Execute Action → Mark as Executed

It prevents execution without confirmation, duplicate confirmation, cross-account confirmation, and unauthorized action access — for example, if a Northstar user creates an escalation action, a LumenWorks user cannot confirm it.

AuthService provides the user context used for authorization (e.g. northstar_customer, lumen_customer, support_agent), determining which account a user belongs to, whether they're an internal support user, and which orders and tickets they can access. This matters because the agent should never expose one customer's information to another.

3. Document and Structured-Data Handling

ParcelPilot uses two types of information. **Structured data** is used for facts requiring precise values — order ID, order status, account ID, ticket status, pickup timestamps, shipment fee, carrier fault, customer fault. For *"What is the status of order ORD-1001?"*, the system retrieves the order directly from the structured dataset rather than searching documents.

Documents are used for policies, contracts, procedures, and product knowledge — whether an order can be cancelled, service credit eligibility, cancellation fees, known product issues, applicable support SLAs. This separation avoids using document retrieval for information that should come directly from the database.

The agent combines both sources when necessary. For *"Can I cancel order ORD-1001 without a cancellation fee?"*, it: retrieves ORD-1001, checks its current status, identifies the customer's account, retrieves the account-specific agreement, retrieves the default cancellation policy, applies source precedence, and generates the answer — an example of multi-step reasoning across both structured data and document retrieval.

4. Source Reliability and Conflict Handling

Not all documents carry equal authority. The system uses a source reliability model based on scope, authority, current/deprecated status, and account relevance, with a simplified precedence order:

Account-Specific Signed Agreement



Current Global Policy / SOP



Current Product Documentation



Deprecated Documentation

For example, the Northstar Logistics Enterprise Agreement states Northstar may cancel any BOOKED shipment before pickup without a cancellation fee, while the default ParcelPilot Cancellation & Service Credit SOP normally applies a fee after 30 minutes. Because the Northstar agreement is account-specific and higher authority, the agent uses the contract-specific rule for Northstar:

Northstar Agreement > Default Cancellation SOP

This lets the agent correctly answer: *"Yes. Northstar Logistics can cancel ORD-1001 without a cancellation fee."* Deprecated sources are given the lowest reliability and never override current policies.

5. Major Technical Trade-Offs

Rule-based intent detection. I used controlled, keyword-based routing instead of a fully autonomous agent, since the dataset supports a limited, known set of workflows (cancellation, service credits, order lookup, ticket lookup, known issues, escalation, confirmation). This gives more predictable behavior, easier debugging, clearer access control, lower implementation complexity, and reduced risk of unsafe actions. A more autonomous LLM-based planner could be added in a future version.

In-memory pending actions. Pending actions are currently managed within the application workflow — sufficient to demonstrate confirmation requirements, action ownership, and duplicate prevention. In production, I would persist pending actions in a database with action ID, user ID, account ID, action type, action payload, status, creation time, expiration time, and audit history.

Local document retrieval. The current retrieval approach is suitable for the assessment dataset. A production version would add better semantic embeddings, metadata filtering before retrieval, hybrid keyword + vector search, reranking, chunk-level citations, and document version management.